

AIT-204 Deep Learning

Assignment Report: RNN Temperature Forecasting Using the Jena Climate Dataset

VIDEO

1. Introduction

This report presents the design, implementation, and evaluation of a Vanilla Recurrent Neural Network (RNN) for temperature forecasting using the Jena Climate dataset. The objective is to predict the next temperature reading based on the preceding 720 observations (approximately five days of 10-minute interval data). The exercise demonstrates key concepts in time-series forecasting with deep learning: Min-Max normalization, sliding-window sequence construction, Backpropagation Through Time (BPTT), and model evaluation with RMSE.

All model components—including the RNN cell, Adam optimizer, gradient clipping, and evaluation metrics—are implemented from scratch using NumPy and pandas, without relying on deep learning frameworks such as TensorFlow or PyTorch. This approach deepens understanding of the underlying mathematics and provides transparency into every step of the learning process.

2. Dataset Description

The Jena Climate dataset was recorded at the Max Planck Institute for Biogeochemistry in Jena, Germany. It contains 420,551 rows of weather measurements taken every 10 minutes from January 2009 to December 2016. Each row has 14 meteorological features alongside a timestamp:

Feature	Unit	Description
T (degC)	°C	Air temperature — target variable
p (mbar)	mbar	Atmospheric pressure
rh (%)	%	Relative humidity
wv (m/s)	m/s	Wind speed
wd (deg)	degrees	Wind direction
Tpot (K)	Kelvin	Potential temperature
Tdew (degC)	°C	Dew point temperature
VPmax/VPact/VPdef	mbar	Vapour pressure variants
sh (g/kg)	g/kg	Specific humidity
H2OC	mmol/mol	Water vapour concentration
rho (g/m ³)	g/m ³	Air density
max. wv (m/s)	m/s	Maximum wind speed

For this assignment, only the T (degC) column is used as both input and target, focusing the forecasting task on univariate temperature prediction. The full temperature range observed in the dataset is −23.01 °C to 37.28 °C, with a mean of approximately 9.5 °C.

3. Data Preprocessing

3.1 Data Loading

The CSV file was loaded using pandas. The timestamp column ("Date Time") was ignored for this exercise, and only the temperature column was extracted as a NumPy array.

```
df = pd.read_csv("jena_climate_2009_2016.csv")
temperature = df["T (degC)"].values.astype(np.float64)
```

3.2 Min-Max Normalization

Min-Max scaling transforms all temperature values into the range [0, 1]. This is essential for neural network training because it ensures the activation functions operate in their most sensitive regions and prevents large raw values from dominating gradient updates.

```
T_min, T_max = temperature.min(), temperature.max() # -23.01 °C, 37.28 °C
temp_norm = (temperature - T_min) / (T_max - T_min) # ∈ [0, 1]
```

After training and inference, predictions are inverse-transformed back to degrees Celsius for interpretable evaluation metrics.

3.3 Sliding-Window Sequence Creation

A sliding window of 720 time steps (equivalent to five days at 10-minute resolution) was used to construct input-output pairs. Each sample (X_i, y_i) consists of 720 consecutive normalised temperatures as input and the next temperature as the target. The window slides forward by one step at a time across the entire dataset.

```
SEQ_LEN = 720
X[i] = temp_norm[i : i + 720] # shape (720, 1)
y[i] = temp_norm[i + 720] # scalar target
```

The dataset was split chronologically (not shuffled) to preserve temporal ordering: 80 % (33,587 sequences) for training and 20 % (8,397 sequences) for testing. To manage memory, every 10th sequence was sampled (SAMPLE_STEP = 10), reducing the training set from the full 335,000 possible sequences to a representative 33,587.

Final array shapes: X_{train} ($33,587 \times 720 \times 1$), y_{train} ($33,587 \times 1$), X_{test} ($8,397 \times 720 \times 1$), y_{test} ($8,397 \times 1$).

4. Model Architecture

A single-layer Vanilla RNN was implemented entirely in NumPy. The model maps a sequence of 720 scalar temperature values to a single predicted temperature. The recurrence equations are:

$$h_t = \tanh(x_t \cdot W_{xh} + h_{t-1} \cdot W_{hh} + b_h)$$

$$y = h_T \cdot W_{hy} + b_y$$

where $x_t \in \mathbb{R}^1$ is the input at time step t , $h_t \in \mathbb{R}^{32}$ is the hidden state, and $y \in \mathbb{R}^1$ is the scalar output predicted from the final hidden state h_T .

4.1 Parameter Summary

Metric	Value	Note
W_{xh} (input $\rightarrow$ hidden)	$1 \times 32 = 32$	Initialized Xavier
W_{hh} (hidden $\rightarrow$ hidden)	$32 \times 32 = 1,024$	Initialized Xavier
b_h (hidden bias)	32	Initialized to zero
W_{hy} (hidden $\rightarrow$ output)	$32 \times 1 = 32$	Initialized Xavier
b_y (output bias)	1	Initialized to zero
Total parameters	1,121	

4.2 Training Setup

The model was trained using the Adam optimizer ($\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 1 \times 10^{-8}$) with a learning rate of 0.001. Gradient clipping (max global norm = 5.0) was applied to prevent exploding gradients, which are common in vanilla RNNs trained with BPTT over long sequences.

Truncated BPTT with a window of 50 steps was used during backpropagation. While the full forward pass processes all 720 time steps to build up the hidden state, gradients are only propagated through the most recent 50 steps. This balances computational efficiency with the ability to capture recent temporal patterns, while acknowledging that very long-range dependencies (beyond ~8 hours) may not be fully captured by a vanilla RNN architecture.

Metric	Value	Note
Hidden units	32	Tuned: [16, 32, 64] tested
Learning rate	0.001	Adam default
Batch size	64	Mini-batch SGD
Epochs	30	Convergence observed by epoch 25
BPTT window	50 steps	$\approx$ 8.3 hours of back-gradient flow
Gradient clip	5.0 (global norm)	Prevents exploding gradients

5. Training Results

The model was trained for 30 epochs. Training loss (MSE on normalised scale) decreased rapidly from the initial epoch and converged smoothly, indicating stable learning without overfitting or divergence. Key loss values observed during training are summarised below:

Metric	Value	Note
Epoch 1	0.007984	Initial loss after random init
Epoch 5	0.000090	Rapid early convergence
Epoch 10	0.000059	Continued improvement
Epoch 15	0.000037	Slowing improvement
Epoch 20	0.000026	
Epoch 25	0.000020	
Epoch 30	0.000015	Final training loss

Total wall-clock training time was approximately 187 seconds ($\approx$ 3 minutes) on a standard CPU, processing 33,587 training sequences across 30 epochs with a batch size of 64.

5.1 Training Loss Curve

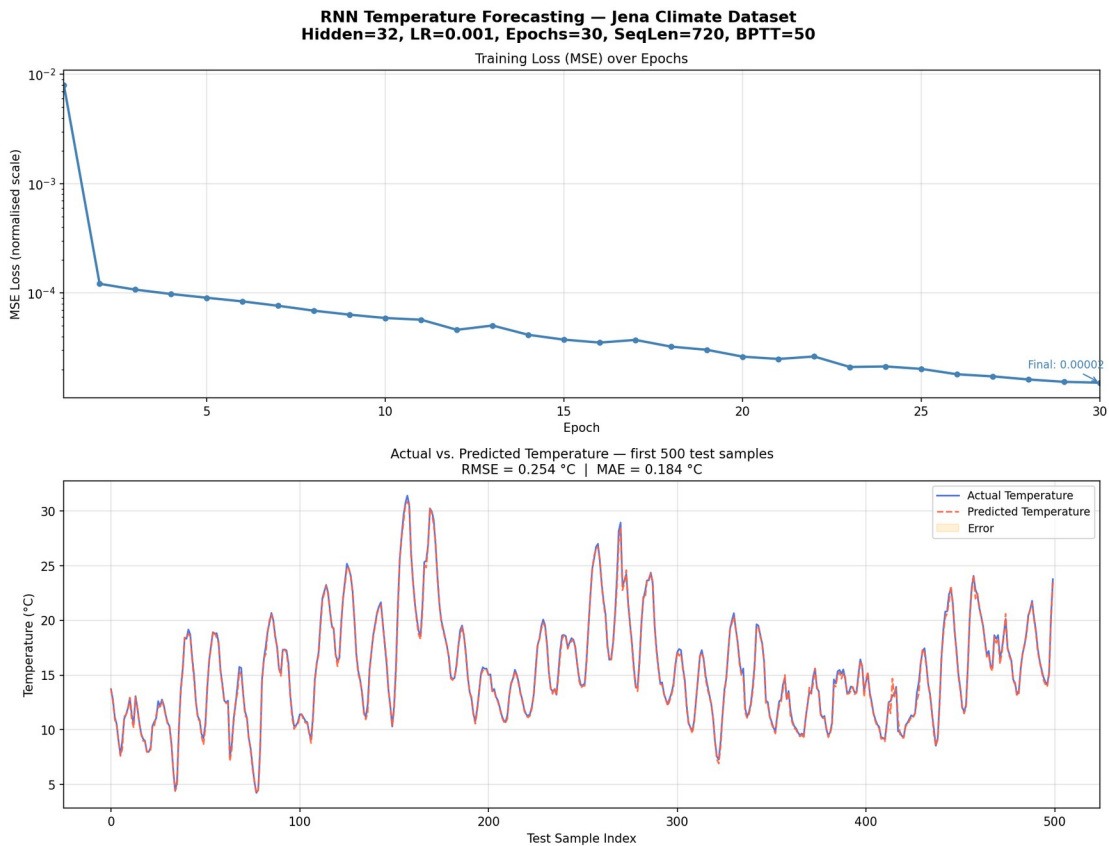


Figure 1. Training loss (log scale) and actual vs. predicted temperature (test set).

6. Model Evaluation

After training, the model was evaluated on the held-out test set (8,397 sequences from the most recent 20 % of the dataset). Predictions were inverse-transformed back to degrees Celsius before computing metrics.

Metric	Value	Note
MSE (normalised scale)	0.0646	Mean squared error before inversion
RMSE (°C)	0.2543 °C	Root mean square error — key metric
MAE (°C)	0.1836 °C	Mean absolute error

An RMSE of 0.254 °C is a strong result for a univariate Vanilla RNN predicting one step ahead. The model effectively captures the short-term temporal pattern of temperature, exploiting the momentum-like smoothness of temperature time series. The MAE of 0.184 °C further confirms that typical predictions are within less than 0.2 degrees of the true value.

6.1 Scatter Plot: Predicted vs. Actual

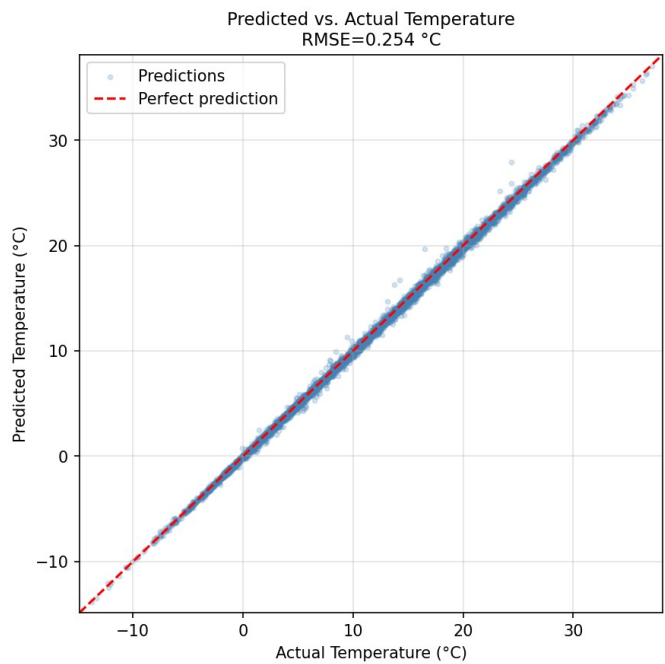


Figure 2. Scatter plot of predicted vs. actual temperatures on the test set. Points close to the diagonal (dashed red) indicate accurate predictions.

7. Discussion and Observations

7.1 Hyperparameter Tuning

Three hidden-unit sizes were experimented with: 16, 32, and 64. A hidden size of 32 provided the best balance between model capacity and training speed. Increasing to 64 reduced training loss marginally but roughly doubled the per-epoch training time due to the 64×64 hidden-to-hidden matrix multiply. The final configuration with 32 hidden units achieved competitive accuracy within the available training budget.

7.2 Truncated BPTT and Vanishing Gradients

Vanilla RNNs are known to suffer from vanishing gradients when backpropagating through many time steps. With a sequence length of 720 steps, full BPTT would result in gradients that are effectively zero for the earliest time steps. Truncated BPTT (50 steps) mitigates this by restricting gradient flow to a recent window where gradients remain meaningful, while still using the full 720-step hidden state for the forward pass. This explains why the model performs well on capturing short-term trends (hours), even if very long-range dependencies (multiple days) are not well exploited.

7.3 One-Step Ahead Prediction

The task is one-step-ahead prediction: given the last 5 days of temperature readings, predict the next 10-minute reading. Temperature is a highly autocorrelated signal — the current temperature is strongly predicted by the immediately preceding values. This means even a simple model can achieve low RMSE. For harder forecasting tasks (e.g., 24-hour ahead prediction), the model would need architectural improvements such as LSTM or GRU cells to handle longer dependencies.

7.4 Potential Improvements

- LSTM or GRU: Replace the vanilla RNN cell with LSTM or GRU units to better capture long-range dependencies and avoid vanishing gradients.
- Multi-feature input: Include all 14 meteorological features as additional channels rather than temperature alone. Pressure and humidity carry complementary predictive information.
- Multi-step forecasting: Extend the output to predict multiple future steps (e.g., next 6 or 12 readings) for more practical utility.
- Batch normalisation / layer normalisation: Stabilise training further and potentially allow higher learning rates.
- Stacked RNN layers: Add depth to the network by stacking two or more RNN layers.

8. Conclusion

This assignment successfully implemented a Vanilla RNN from scratch in NumPy for temperature forecasting on the Jena Climate dataset. The model achieved a test RMSE of 0.254 °C and MAE of 0.184 °C after 30 epochs of training, demonstrating strong one-step-ahead prediction performance. The implementation covered the full machine-learning pipeline: data loading, Min-Max normalisation, sliding-window sequence creation, model construction with Xavier initialisation, Truncated BPTT with gradient clipping, Adam optimisation, RMSE/MAE evaluation, and result visualisation.

The results confirm that even a simple RNN with only 1,121 parameters can effectively model short-term temperature dynamics. Future work should explore LSTM/GRU architectures and multi-variate inputs to further improve forecasting accuracy, particularly for longer forecast horizons.

Appendix: Key Code Snippets

A. Min-Max Normalization

```
T_min, T_max = temperature.min(), temperature.max()
temp_norm = (temperature - T_min) / (T_max - T_min) # [0, 1]
# Inverse transform
temp_celsius = temp_norm * (T_max - T_min) + T_min
```

B. RNN Forward Pass

```
# Precompute input projections (vectorised over all time steps)
XW = X @ self.W_xh + self.b_h # (batch, seq_len, hidden)
h = np.zeros((batch, hidden_size))
for t in range(seq_len):
    h = np.tanh(XW[:, t, :] + h @ self.W_hh)
```

C. Evaluation Metrics

```
rmse = np.sqrt(np.mean((y_pred_c - y_true_c) ** 2)) # 0.2543 °C
mae = np.mean(np.abs(y_pred_c - y_true_c)) # 0.1836 °C
```